

Міністерство освіти і науки України
Національний університет «Запорізька політехніка»

Кафедра програмних засобів

ЗВІТ

Дисципліна «Безпека та захист програм і даних»

Робота №3

Тема «Стеганографічний захист»

Виконав варіант 19

Студент КНТ-122

Онищенко О.А.

Прийняли

Викладач

Качан О.І.

МЕТА

Ознайомитися з методами стеганографічного захисту інформації

ЗАВДАННЯ

Реалізувати метод LSB у пакеті Visual C++. В якості контейнера застосувати файл формату *.bmp. В якості повідомлення використати своє «Прізвище Ім'я По-батькові».

КОНТРОЛЬНІ ПИТАННЯ

Чим відрізняється стеганографія від криптографії?

Головна відмінність полягає у **меті** захисту інформації:

Характеристика	Криптографія	Стеганографія
Мета	Приховати зміст повідомлення.	Приховати сам факт існування повідомлення.
Результат	Повідомлення виглядає як набір випадкових символів ("сміття"), що привертає увагу, але не читається без ключа.	Повідомлення виглядає як звичайний файл (фото, музика), який не викликає підозр.
Аналогія	Лист у сейфі або зашифрований шифром Цезаря.	Лист, написаний невидимим чорнилом між рядків звичайної газети.

Що таке стегоконтейнер?

Стегоконтейнер (або просто контейнер) — це файл-"обгортка", який використовується для приховування секретних даних. Це цілком

звичайний файл, який можна відкрити та переглянути, і він не викликає підозр у стороннього спостерігача.

- **Порожній контейнер:** Файл до внесення в нього секретних даних.
- **Заповнений контейнер** (стегофайл): Файл, що вже містить приховану інформацію.

Найчастіше контейнерами виступають:

- Цифрові зображення (BMP, JPEG, PNG).
- Аудіофайли (WAV, MP3).
- Відеофайли.

В чому сутність метода LSB?

LSB (Least Significant Bit — Найменш значущий біт) — це найпопулярніший метод стеганографії, суть якого полягає у заміні останніх (найменш важливих) бітів у байтах файлу-контейнера на біти секретного повідомлення.

Як це працює (на прикладі зображення):

Кожен піксель зображення кодується числами (наприклад, для кольору RGB це три байти).

Якщо змінити останній біт у байті (наприклад, з 11111110 на 11111111), числове значення кольору зміниться мінімально (наприклад, з 254 на 255).

Результат: Людське око фізично не здатне помітити таку мізерну різницю у відтінку кольору, але комп'ютер може витягти ці змінені біти та скласти з них прихований текст або файл.

Перевага: Висока ємність (можна сховати багато даних) та простота реалізації. **Недолік:** Слабкий захист від стиснення або редагування файлу (секрет може зруйнуватися).

ВИКОНАННЯ

Процес роботи описано рисунками нижче:

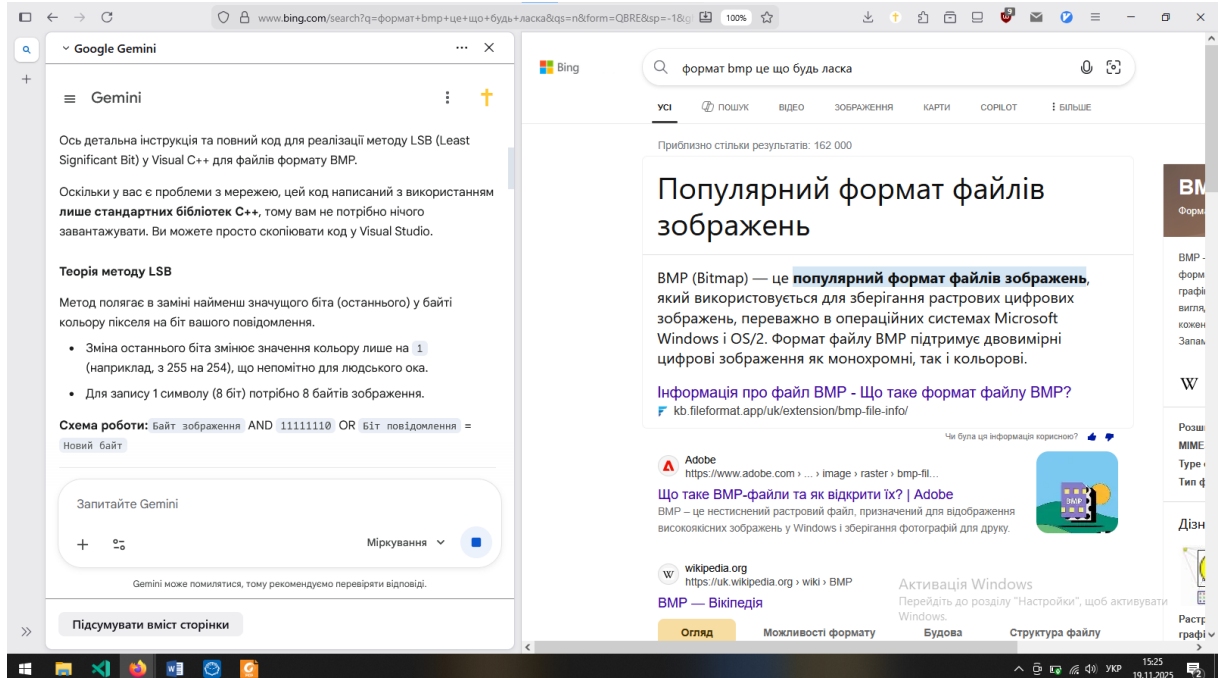


Рисунок 1.1 – Початок роботи

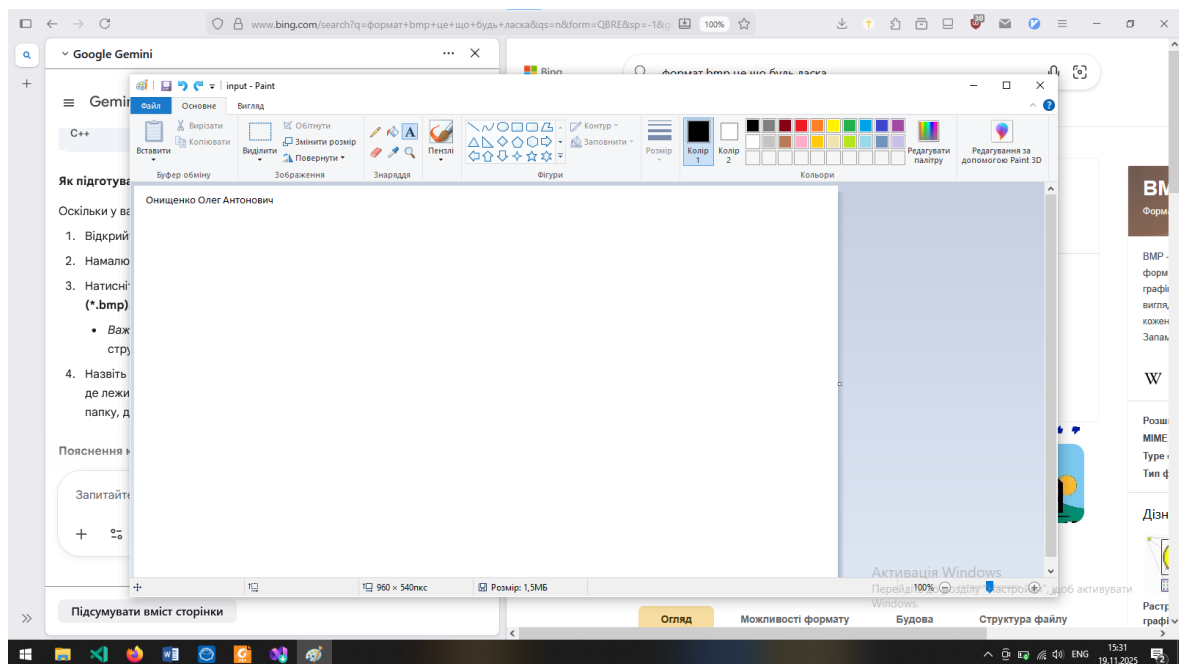


Рисунок 1.2 – Створення bmp зображення в Пейнті

Для економії місця програму зроблено на Пайтоні.

Код

```
1. import os
2.
3. def hide_lsb(input_path, output_path, text_message):
4.     try:
5.         # Відкриваємо файл для читання бінарних даних
6.         with open(input_path, 'rb') as f:
7.             bmp_header = f.read(54) # Перші 54 байти - це
заголовок
8.             image_data = bytearray(f.read()) # Решта - пікселі
9.
10.            # Додаємо спеціальний стоп-символ (null-byte), щоб знати
де кінець тексту
11.            message_bytes = text_message.encode('utf-8') + b'\x00'
12.
13.            # Перевірка: чи вистачить місця (1 байт повідомлення = 8
байт картинки)
14.            if len(message_bytes) * 8 > len(image_data):
15.                print("Помилка: Зображення занадто мале для такого
тексту!")
16.                return
17.
18.            data_idx = 0 # Індекс байта картинки
19.
20.            # Проходимо по кожному байту нашого повідомлення
21.            for byte in message_bytes:
22.                # Проходимо по кожному біту цього байта (від 0 до 7)
23.                for bit_pos in range(8):
24.                    # Отримуємо значення біта (0 або 1)
25.                    # зсуваємо байт і беремо останній біт
26.                    bit = (byte >> bit_pos) & 1
27.
28.                    # Змінюємо байт картинки:
29.                    # & 254 (11111110) - очищає останній біт
30.                    # | bit - записує наш біт
31.                    image_data[data_idx] = (image_data[data_idx] &
254) | bit
32.
```

```

33.             data_idx += 1
34.
35.             # Записуємо результат у новий файл
36.             with open(output_path, 'wb') as f:
37.                 f.write(bmp_header)
38.                 f.write(image_data)
39.
40.             print(f"[OK] Повідомлення успішно сховано у
'{output_path}')"
41.
42.         except FileNotFoundError:
43.             print(f"Помилка: Файл '{input_path}' не знайдено.")
44.         except Exception as e:
45.             print(f"Сталася помилка: {e}")
46.
47.     def decode_lsb(image_path):
48.         # Функція для перевірки (читання повідомлення назад)
49.         try:
50.             with open(image_path, 'rb') as f:
51.                 f.read(54) # Пропускаємо заголовок
52.                 image_data = f.read()
53.
54.                 extracted_bytes = bytearray()
55.                 current_byte = 0
56.                 bit_count = 0
57.
58.                 for byte in image_data:
59.                     # Витягуємо останній біт з байта картини
60.                     bit = byte & 1
61.
62.                     # Записуємо біт у наш поточний байт
63.                     # (bit << bit_count) зсуває біт на потрібну позицію
64.                     current_byte |= (bit << bit_count)
65.
66.                     bit_count += 1
67.
68.                     # Коли зібрали 8 біт (1 повний байт символу)
69.                     if bit_count == 8:
70.                         # Якщо зустріли нульовий байт - кінець
повідомлення
71.                         if current_byte == 0:

```

```

72.                 break
73.
74.                 extracted_bytes.append(current_byte)
75.                 current_byte = 0
76.                 bit_count = 0
77.
78.                 print(f"[Прочитано] Зміст повідомлення:
{extracted_bytes.decode('utf-8')}")
79.
80.             except Exception as e:
81.                 print(f"Помилка читання: {e}")
82.
83. # --- ГОЛОВНА ЧАСТИНА ---
84. if __name__ == "__main__":
85.     input_file = "input.bmp"
86.     output_file = "key.bmp"
87.
88.     # Поставити поточну папку як стандартний шлях
89.     os.chdir(os.path.dirname(__file__))
90.
91.     # ВАШЕ ПРИЗВИЩЕ ІМ'Я ПО-БАТЬКОВІ
92.     my_message = "Онищенко Олег Антонович"
93.
94.     print("--- Python LSB Steganography ---")
95.
96.     # 1. Ховаємо текст
97.     hide_lsb(input_file, output_file, my_message)
98.
99.     # 2. Для перевірки одразу пробуємо прочитати з нового файлу
100.    if os.path.exists(output_file):
101.        decode_lsb(output_file)

```

Результати

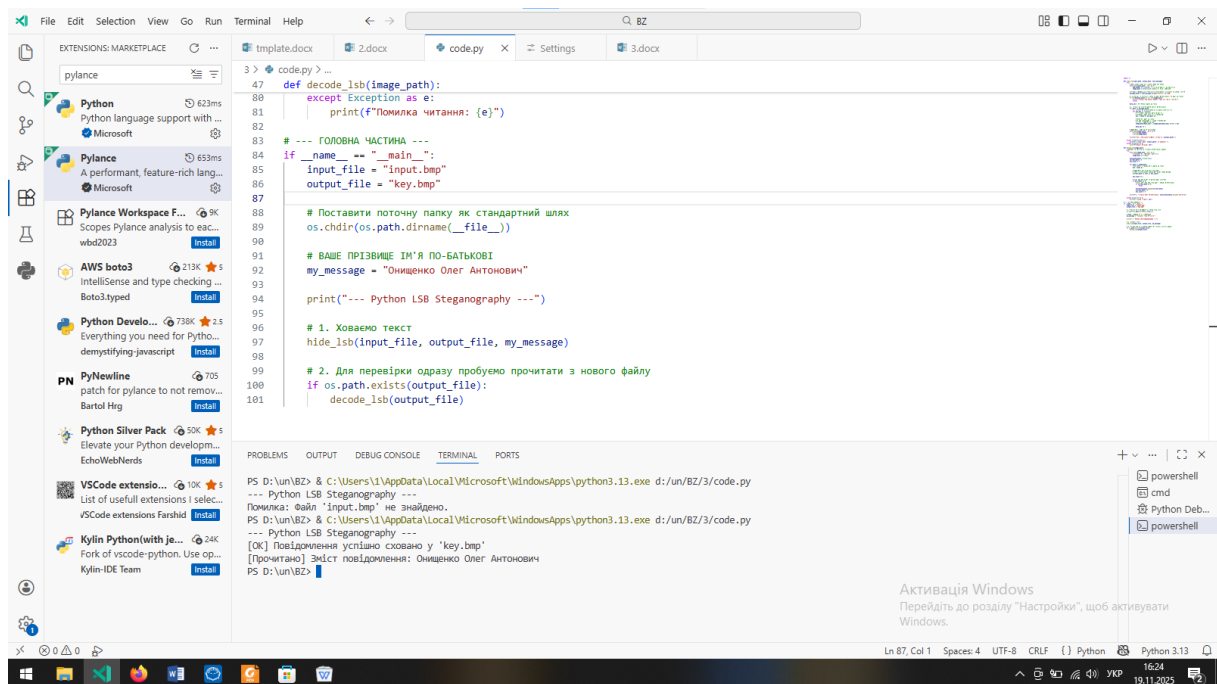


Рисунок 2.1 – Результати виконання

ВИСНОВКИ

Слава Богу за Його невимовний дар. Роботу зроблено до кінця. Головне пам'ятати, що немає нічого схованого, що не вийшло б на яв, ані прихованого, що не явилось б. Бо день Господень виявить таємні думки людей.